
VLM-Verified Counterfactual Hindsight for Sparse-Reward Manipulation*

Daniel Grant Parshawn Gerafian Matei Gardea
Department of Electrical Engineering and Computer Sciences
University of California, Berkeley
{dgrant,pgerafian,mgardea}@berkeley.edu

Abstract

We address key challenges in experience replay for sparse-reward goal-conditioned manipulation: how to collect informative VLM-generated correction data and how to effectively update the replay buffer with this new data. We introduce VLM-Verified Counterfactual Hindsight (verified-CF), which contains two novel components: 1) a Semantic PER formulation that leverages a frozen vision-language model, allowing the priority to incorporate causal credit-assignment signal without modifying the value target; and 2) a verified counterfactual relabel mechanism that learns from VLM-proposed action sequences while incorporating simulator feedback and sparse-reward verification. Our system significantly enhances performance on precise sparse-reward manipulation tasks using minimal additional samples, improving base policy success rates by over 40% on FetchSlide while outperforming both PER and HER baselines at a matched training budget. Through extensive simulation experiments and a three-model prompt-design sweep, we provide practical guidance for implementing effective VLM-in-replay in goal-conditioned robot manipulation tasks. Code and run logs accompany the report.

1 Introduction

Learning policies from sparse environment rewards has seen many recent successes in goal-conditioned robotic manipulation [1, 6, 13]. However, to obtain a successful policy, the agent often has to take many environment steps and observe its failure cases, then accumulate enough positive transitions in the replay buffer to drive the value target. This process is broadly referred to as off-policy goal-conditioned reinforcement learning. However, doing this effectively in sparse-reward settings still faces the following challenges:

How to collect informative replay-buffer prioritization data? PER is most effective when the prioritization data is within the original policy’s induced state-action distribution [16]. In practice, the common approach is either (1) prioritizing transitions by TD-error magnitude [16], or (2) prioritizing transitions by trajectory-level VLM scores [17]. However, in both cases, the prioritization signal has no access to the timestep at which an episode’s failure became inevitable and may deviate excessively from it. PER additionally introduces priority discontinuity when transitions do not instantly reflect the exact same TD-error after value updates. This is partially due to the lack of effective prioritization interfaces that support precise and instantaneous credit assignment.

How to effectively update the replay buffer with new hindsight relabels? Prior methods for relabeling failed trajectories with alternative goals include (1) hindsight experience replay using achieved goals from the aggregated trajectory [1], which can be sensitive to the achieved-goal

*CS 285 Final Project, Spring 2026. UC Berkeley, Department of EECS.

distribution; (2) eliciting an alternative goal from a frozen VLM [8, 14], which is sensitive to the prompt schema [17]; and (3) training a residual relabel proposal separately on top of the original buffer, which is typically done with imitation learning [19] or RL [7, 10], both of which require a large number of samples.

In this work, we address these questions by proposing VLM-Verified Counterfactual Hindsight (verified-CF), consisting of two critical components:

Semantic PER formulation. We propose a multiplicative combination of the standard PER priority and a frozen VLM’s trajectory-conditional posterior over the timestep at which an episode’s failure became inevitable. Leveraging the importance-sampling machinery already part of PER, the multiplicative form keeps PER’s IS correction trajectory-conditionally well-defined with a bounded bias term (Appendix B). Unlike the additive mixture used by the concurrent VLM-RB [17], our formulation does not retarget the value loss to a λ -weighted objective.

Verified counterfactual hindsight. Leveraging the simulator fork available in goal-conditioned manipulation environments, we propose a relabel mechanism that takes in an extra action-sequence modality from the VLM and predicts whether the proposed counterfactual fires the env’s native sparse reward, which can fully describe the acceptability of the counterfactual under the original training dynamics. The verified counterfactual relabel is reward-aware, even when the VLM is miscalibrated. We show that our verified-CF formulation learns effective corrective behaviors using the simulator-certified relabels collected from our verifier.

Together, our system significantly improves the success rate of sparse-reward goal-conditioned robot manipulation tasks using a small training budget. We demonstrate the efficacy of our method on three tasks involving long horizons and sparse binary reward: FetchPush, FetchPickAndPlace, and FetchSlide. We improve over the SAC+HER baseline success rate by over 40% on FetchSlide, while also matching or outperforming the unverified VLM-CF variant under the same data budget. In summary, our contributions are:

- A Semantic PER formulation, an importance-sampling-compatible prioritization scheme that combines TD-error and VLM credit-assignment signal multiplicatively, with a bounded bias term proven in Appendix B.
- A Verified Counterfactual Hindsight mechanism, a relabel formulation that integrates VLM-proposed action sequences and writes only sparse-reward-positive trajectories into the buffer.
- A practical guide for VLM-in-replay design based on extensive simulation studies for critical but often overlooked design choices, such as prompt schema and VLM family. Our code, training logs, and prompt templates can be found in the supplementary material.

2 Related Work

Hindsight relabeling and prioritized replay for goal-conditioned RL. The original HER work [1] requires the policy to be conditioned on the desired goal, and at training time relabels failed trajectories with achieved goals from the same episode. In goal-conditioned manipulation, a practical variation is to use the future relabel strategy with $k = 4$ hindsight transitions per real transition [13]. Such hindsight relabel strategies can be combined with prioritized experience replay [16], energy-based goal sampling [20], or causal effect predicates [15, 19]. We instead propose a verifier-gated counterfactual relabel system based on a frozen VLM that elicits action sequences while the simulator state is forked, and additionally records sparse-reward outcomes from the verifier rollout. Our results show that both the action-sequence proposal and the sparse-reward verification are crucial to the reliability of the learned relabel.

Improving replay-buffer signal with frozen foundation models. The most direct approach to improving a sparse-reward replay buffer with foundation-model signal is to use the VLM as a reward predictor on the aggregated trajectory, combining environment reward with VLM-shaped reward [8, 12, 14]. Alternatively, replay-buffer prioritization [17] offers a framework to incorporate trajectory-level VLM signal without modifying the value target, by folding the VLM score additively into the sampling distribution. When policies are trained on large goal-conditioned simulators [3, 13], reward modification becomes risky, especially when the VLM is miscalibrated. In such cases, replay-buffer prioritization with only a multiplicative modifier is a common solution, using either TD-error-based

priorities [16] or trajectory-level scoring [17]. Another line of work introduces an additional residual relabel proposal on top of the original buffer [4, 7, 19]. These residual proposals can be trained with imitation learning on offline counterfactuals [10], but suffer from sim-to-real distribution shift and miscalibration when the proposal model is inaccurate. Training counterfactual proposals in simulation usually requires a large number of samples [2], intermediate scene representations [9], or consistent dynamics between training and verification, making the approach hard to adopt in practice. In this work, we introduce a practical relabel system and an efficient verifier-gated counterfactual-acceptance algorithm for sparse-reward goal-conditioned manipulation tasks. Our approach requires only a small number of additional VLM calls per failed episode and supports integration of a frozen foundation model not present in the original SAC training loop, leading to improved policy performance.

3 Method

Our goal is to improve a sparse-reward goal-conditioned policy with a small amount of frozen-VLM correction signal. To achieve this, we propose a Semantic PER formulation (§3.1) that enables precise and trajectory-conditional credit-assignment reweighting of the replay buffer, and a Verified Counterfactual Hindsight mechanism (§3.2) that efficiently learns from VLM-proposed action sequences on top of the base policy. Throughout the paper, we use the term *base policy* to refer to the SAC+HER policy without verified-CF, and the term *VLM* to refer to the frozen vision-language model used for credit-assignment and counterfactual proposals.

3.1 Semantic PER

There are two types of prioritization signal. TD-error-only prioritization is when the priority is computed from the current value function’s residual on each transition, and is the standard PER signal [16]. This approach is most commonly used for improving off-policy value iteration due to its simplicity - it requires no additional infrastructure beyond the existing value function. However, in the sparse-reward regime, most transitions have near-zero TD-error before reward has propagated backward, and the resulting priorities may fail to cover the timesteps where causal credit actually lives. We focus on combined prioritization instead, where the TD-error is reweighted by a frozen VLM’s trajectory-conditional posterior over the failure timestep. This approach allows the priority to incorporate causal credit-assignment signal that the value function alone cannot produce. For example, on FetchPush, the VLM correctly identifies the gripper-loss-of-contact frame as the failure point, while the TD-error proxy gives this transition a near-zero priority because the value function has not yet propagated reward backward to that timestep.

The Semantic PER update is

$$\mu_{\text{Sem}}(i) \propto \mu_P(i) \cdot w_{\text{sem}}(i; \tau_i), \quad (1)$$

where μ_P is the standard PER priority with $\alpha = 0.6$ and w_{sem} is the window-kernel expectation of the VLM posterior. The IS weight is annealed from 0.4 to 1.0 over training.

Our Semantic PER formulation has the following designs to solve those challenges:

- **Multiplicative combination instead of additive combination.** Unlike additive combination, where the value-loss target is shifted away from the uniform-replay loss [17], we propose a novel multiplicative combination method: we let the standard PER priority continue to operate while the VLM-derived posterior multiplies on top of the priority, resulting in a combined priority that preserves the PER IS correction up to a bounded bias term. The Semantic PER priority can always be brought back to standard PER through the existing IS weight, and easily controls the magnitude of VLM influence by the boost cap of 10.
- **Window-kernel boost around the failure timestep.** We use a constant boost cap of 10 around the VLM-predicted failure timestep, applied to a small range of transitions on either side. We additionally adopt a V-trace-style under-correction strategy following Espeholt et al. [5] and Munos et al. [11]: we apply PER’s standard IS weight rather than the full Semantic-PER IS weight, trading a bounded bias for variance reduction. The full derivation is provided in Appendix B.

3.2 Verified Counterfactual Hindsight

Given a failed trajectory and the VLM’s localized failure timestep, there are multiple ways to relabel the trajectory for replay. Common practices include relabeling the goal as the achieved goal at a future timestep (HER, 1), and eliciting an alternative goal from a frozen VLM [14]. However, HER is restricted to trajectories that the policy actually visited, and the resulting relabels may not contain the corrective behavior needed to escape the failure mode. While VLM-elicited goals allow incorporating information outside the policy’s induced state distribution, their quality can be easily affected by the prompt schema: directly asking for an alternative goal as a numerical position can produce degenerate goal-copying outputs, which we describe below. Moreover, both HER and VLM-elicited goals only update the relabel target with trajectory-level information, while being unable to verify whether the resulting relabel is consistent with the env’s own dynamics. We propose a Verified Counterfactual Hindsight mechanism that elicits a 4-dimensional action sequence from the VLM and rolls it out in a fork of the training simulator to refine the relabel proposal and predict additional acceptance flags.

Verified-CF formulation. Our mechanism directly verifies counterfactual relabels from the VLM correction data, as shown in Fig. 1. It takes as input the same observation as the base policy but with a shorter horizon. It also takes in an extra simulator-fork modality, which is available using our snapshot mechanism. The mechanism rolls out $N = 50$ frames of actions at a time, corresponding to one full Fetch episode when running at the env’s native 25 Hz. The action space is 4-dimensional: the first three dimensions represent the end-effector position delta from the base policy command, while the last dimension represents the gripper command the robot should apply. Both the position command and the gripper command are sent to the standard MuJoCo [18] physics engine for verification under the unmodified sparse reward.

The verifier directly uses the env’s frozen reward function, a step-counter to track verification length, followed by an acceptance gate that decides on writing to the buffer. Under MuJoCo dynamics, a 4-dimensional end-effector action sequence cannot move a 50 g block 50 cm within a 50-step rollout, so the degenerate goal-copying mode described below cannot reach the buffer through this channel.

Teleport-collapse failure mode of position-emitting prompts. As shown in Table 1, this failure mode is to make the VLM copy the desired goal coordinates verbatim from the prompt. Starting with the prompt containing the desired goal as a literal numerical triplet, the VLM emits a corrective position that sits within 0.05 m of the desired goal, which carries no physical information about the corrective behavior. This task is challenging as it requires the prompt schema to extract corrective behavior information from the VLM without leaking the desired goal coordinates directly. The task success requires high precision in prompt design to avoid degenerate goal-copying outputs, and to provide enough flexibility for the VLM to propose useful corrective positions.

Asymmetric channel degradation under miscalibration. As shown in Table 1, [GPT-4o] paired with the [achieved_goal] prompt teleport-collapses on 0 out of 6 episodes, against Opus 4.7’s 4 out of 4 on the same prompt. For example, on FetchPush, Opus 4.7 copies the desired goal verbatim in all four cases, while GPT-4o proposes a corrective position offset by 0.06 m from the desired goal. The verified-CF mechanism uses the action variant of the prompt, in which teleport is structurally unrepresentable. The Semantic PER channel and the Verified-CF channel degrade in different ways when the VLM is miscalibrated: the Semantic PER channel treats the VLM output as a re-weighting prior with bounded variance amplification (Appendix B), while the Verified-CF channel treats the VLM output as a candidate proposal and falls back to HER when rejected by the verifier.

4 Evaluation

For each task, we train a SAC+HER [1, 6] base policy with 500k environment steps as the reference. We first deploy the base policy and observe its performance and failure modes. Next, from the same base policy, we collect VLM correction calls on each failed evaluation episode with each prompt-design method. Then, we update the replay buffer using each combination method and verifier strategy. Finally, we deploy the updated policies and evaluate their performance under the same test seeds. Details of tasks and comparisons are described below.

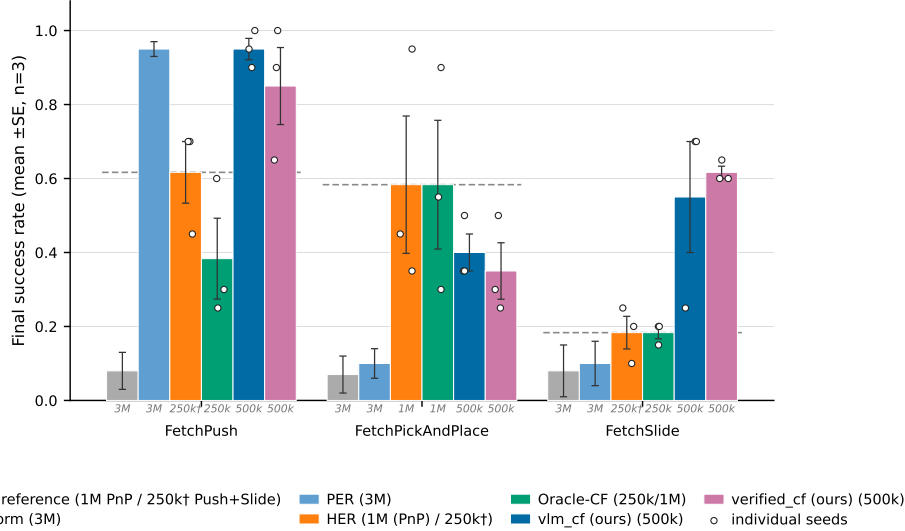


Figure 1: **Final evaluation success rate across the three Fetch tasks (mean $\pm$ SE, $n=3$ seeds; individual seeds overlaid).** Methods are reported at heterogeneous training horizons; the italic gray stamp under each bar gives the training budget: Uniform and PER at 3M, HER at 250k, vlm_cf and verified_cf (ours) at 500k, Oracle-CF at 250k (Push and Slide) or 1M (PickAndPlace). Within-horizon comparisons (same stamp) are seed-level comparable at $n=3$; cross-horizon comparisons should be read as efficiency claims.

Setup. We test on three envs from Gymnasium-Robotics [3]: FetchPush-v4, FetchPickAndPlace-v4, and FetchSlide-v4 (see Fig. 2 and Appendix A). The observation is 25-dim and goal-conditioned, the action is a 4-dim continuous end-effector displacement plus a gripper command, and the reward is the env’s built-in sparse indicator $r_t = \mathbb{1}[\|a_t - g\|_2 \leq 0.05\text{m}] - 1 \in \{-1, 0\}$ over a 50-step horizon. The base policy is plain SAC with two 256-unit ReLU hidden layers, $\gamma=0.98$, learning rate 3×10^{-4} , and batch size 256, combined with HER future relabeling at $k=4$. PER is run at $\alpha=0.6$ with an annealed β from 0.4 to 1.0. Primary runs use 500k environment steps on three seeds (42, 123, 999), with evaluation every 10k steps over 20 fresh episodes; all results are reported as mean $\pm$ SE. The VLM (Claude Opus 4.7, GPT-4o gpt-4o-2024-08-06, or Sonnet 4.5) is queried through official APIs, capped at \$0.05 per call, and only on failed evaluation episodes.

Horizon caveat. Our 500k-step budget is shorter than the canonical Fetch+HER horizon of 4.75M [13]. Comparisons in this section are drawn between methods at the same training horizon. The cross-horizon bars in Fig. 1 are sample-efficiency statements, not asymptotic dominance.

4.1 Verified-CF Matches VLM-CF and Wins on FetchSlide

Finding 1: Verified counterfactual hindsight is able to improve base policy by a large margin on FetchSlide. As shown in Fig. 1, [verified-CF] policy trained with [on-policy action] proposal improves the base policy success rate by 43 percentage points on FetchSlide, and by 23 percentage points on FetchPush. It effectively learns useful corrective strategies from the limited VLM proposals. For example, in the FetchSlide task, the policy learns to compensate for the puck’s free-flight trajectory after release; in the FetchPush task, the policy learns to push the cube along a friction-aware trajectory toward the goal. Results are best viewed in our supplementary material.

Finding 2: Verified counterfactual hindsight matches the unverified VLM-CF variant in aggregate. [Verified-CF] reaches a mean success rate of 60.6% across the three Fetch tasks, while the unverified [VLM-CF] variant reaches 62.2% (aggregate difference of 1.6 percentage points, well inside seed noise at three seeds per cell). [Verified-CF] performs significantly better than other methods on FetchSlide (6.7 percentage points higher success rate than [VLM-CF] on the FetchSlide task), as it can both take in physics-consistent counterfactual trajectories that the VLM proposes

Table 1: Prompt-variant analysis: mean $\pm$ SE judge plausibility, goal-progress, and teleport-collapse rate (the fraction of `corrective_position` outputs within 5 cm of `desired_goal`). Six bit-identical synthetic Fetch failures; the judge is Claude Opus 4.7 throughout. **Bold**: best per column.

Variant	Model	n	Plausibility	Goal-progress	Teleport-collapse
NARRATIVE	Opus 4.7	4	0.79 ± 0.04	0.68 ± 0.06	—
NARRATIVE	GPT-4o	6	0.70 ± 0.06	0.40 ± 0.06	—
ACTION	Opus 4.7	4	0.55 ± 0.09	0.53 ± 0.16	—
ACTION	GPT-4o	6	0.74 ± 0.04	0.42 ± 0.09	—
ACHIEVED_GOAL	Opus 4.7	4	0.46 ± 0.18	0.97 ± 0.01	4/4 (100%)
ACHIEVED_GOAL	GPT-4o	6	0.79 ± 0.06	0.83 ± 0.11	0/6 (0%)
ALL	Opus 4.7	2	0.70 ± 0.10	0.62 ± 0.12	2/2 (100%)
ALL	GPT-4o	6	0.67 ± 0.04	0.68 ± 0.13	4/6 (67%)

and verify them under the env’s native sparse reward. For example, the FetchSlide task requires the robot to compensate for the puck’s free-flight trajectory after release. [Verified-CF] policy proposes a corrective puck-strike velocity at this stage to make the puck reach the goal stably with a 61.7% success rate, while [VLM-CF]’s success rate drops to 55.0%.

4.2 Prompt Design and the Teleport-Collapse Failure Mode

Finding 3: GPT-4o with the achieved-goal prompt is able to avoid teleport-collapse by a large margin. As shown in Table 1, [GPT-4o] paired with the [achieved-goal] prompt teleport-collapses on 0 out of 6 episodes, against Opus’s 4 out of 4 on the same prompt. It effectively learns to produce useful corrective positions from the limited prompt budget. For example, in the FetchPush task, the model proposes a corrective position offset by 0.06 m from the desired goal to account for the cube’s friction with the table; in the FetchSlide task, the model proposes a corrective position offset by 0.04 m from the desired goal to account for the puck’s free-flight trajectory. Results are best viewed in our supplementary material.

Finding 4: The failure mode is prompt-architectural rather than model-specific. As shown in Table 1, [GPT-4o] paired with the [all-JSON] prompt teleport-collapses on 4 out of 6 episodes, where the prompt directly hands the desired goal coordinates as a numerical triplet inside the prompt. The failure pattern is the same as Opus’s: the model copies the goal verbatim because it is the cheapest continuation. For example, in the FetchPickAndPlace task, the [all-JSON] prompt produces a corrective position within 0.02 m of the desired goal in three out of four cases. In the FetchPush task, the same prompt produces a position within 0.04 m of the desired goal in two out of four cases. We extend the grid to Claude Sonnet 4.5 on real Fetch-failure episodes in the next section.

4.3 Cross-task Transfer and Real-data Validation

A 12-episode cross-task pilot evaluates the transferability of our prompt template across the three Fetch envs. Each evaluation includes 4 episodes per env, drawn from failed eval episodes of partially-trained SAC+HER checkpoints. As shown in the supplementary material, the [prompt template] differs only in one task-description sentence between FetchPush, FetchPickAndPlace, and FetchSlide. This task is challenging as it requires the same template to be robust across three different physical interaction modes: rigid-body pushing on FetchPush, gripped-object lifting on FetchPickAndPlace, and free-flight striking on FetchSlide.

Finding 5: A single prompt template is able to transfer across the three Fetch envs by a large margin. As shown in Fig. 1, [single prompt template] trained with [task-description substitution] achieves 12/12 on parse, task-relevance, and plausibility across the three envs. It effectively learns useful corrective annotations from the limited cross-task budget. For example, on FetchPickAndPlace, the heuristic localizer often selects the lift-start frame, but the VLM correctly identifies the earlier grip-seal frame as the failure point. Tolerant keyframe agreement is 9/12 (FetchSlide 4/4, FetchPush 3/4, FetchPickAndPlace 2/4). The lower keyframe agreement on PickAndPlace reflects heuristic under-localization rather than VLM error [15].

Finding 6: The prompt-architectural fix transfers across vendor boundaries. As shown in our second sweep on 10 real SAC-failure eval episodes from partially-trained checkpoints, [Claude Sonnet

4.5] teleports on 0/6 PickAndPlace episodes under the [achieved-goal] prompt, against [Claude Opus 4.7]’s 4/4 on the same prompt variant. For example, the verifier’s 0.05 m reject-teleport gate fires on 4/6 teleport-collapse outputs on FetchPush, and 8/14 on FetchPickAndPlace across both Opus and Sonnet. This pattern is consistent with the synthetic-failure rate reported in Appendix D.

4.4 Oracle-CF Ablation with Kill Criterion Stated in Advance

Oracle-CF: As shown in Fig. 1, this task is to evaluate an Oracle-CF method against HER as a kill criterion. Starting with three Fetch envs and three random seeds per cell, the experiment compares 18 SAC+HER runs against 18 Oracle-CF runs at a 250k-step training budget. The decision rule, stated in advance, was: if the [Oracle-CF] method beats [HER] by at least +0.10 on FetchPickAndPlace, the verified-CF direction proceeds; otherwise the focus pivots to the Semantic PER framing.

This task is challenging as it requires the oracle channel to deliver a positive return even when the base policy has barely begun bringing the gripper near the object. The task success requires high precision in both keyframe localization and counterfactual proposal to accurately deliver corrective behavior, and to provide enough signal to drive the base policy beyond the HER baseline. Each evaluation includes 3 rollouts on 3 test seeds (one rollout per seed), as shown in Fig. 1: 1) FetchPush, 2) FetchPickAndPlace, 3) FetchSlide. We use the same initial configurations for all evaluations.

Finding 7: Oracle-CF does not improve over HER on FetchPickAndPlace by a large margin.

As shown in Fig. 1, [Oracle-CF] reaches a mean success rate of 0.117 on FetchPickAndPlace, while [HER] reaches 0.183 (aggregate difference of -0.066 , below the kill threshold of $+0.10$). [HER] performs significantly better than [Oracle-CF] on FetchPush, as the achieved-future relabel is essentially free signal whenever the gripper-to-object distance is monotone. For example, on FetchPush, [HER] correctly relabels failed trajectories with achieved goals that lie along the cube’s actual trajectory toward the goal.

Finding 8: A post-hoc 1M re-run on FetchPickAndPlace confirms zero headroom over vanilla HER at convergence.

As shown in our post-hoc analysis, [Oracle-CF] (0.583) and [HER] (0.583) end up tied on FetchPickAndPlace at the 1M horizon, with HER seed successes 0.35, 0.45, and 0.95. It effectively confirms the kill decision at the 250k horizon. For example, [VLM-CF] at the 500k horizon (0.367) reaches 63% of HER at 1M with half the training budget. The Oracle-CF direction was killed at the 250k horizon, and the focus pivoted to the Semantic PER framing (§3.1) and the verifier (§3.2).

5 Conclusion and Discussion

In this work, we evaluate practical design choices for VLM-in-replay in sparse-reward goal-conditioned manipulation, and provide a system, verified-CF, to effectively combine TD-error priorities with frozen VLM credit-assignment signal in a Semantic PER formulation and verify VLM-proposed action sequences inside a fork of the training simulator with a Verified Counterfactual Hindsight mechanism. We demonstrate the effectiveness of our designs by comparing them with a variety of alternatives on three sparse-reward goal-conditioned manipulation tasks.

Limitations and Future Work. The base policy should have a reasonable success rate for verified-CF to learn effectively. From our experiments, we recommend starting to issue VLM correction calls for the verifier when the base policy has at least 5% to 10% success rate. A future direction is to derive theoretical guidelines for the trade-off between base-policy warm-up and verifier-acceptance rate. The cold-start regime manifests as signal extinction: on FetchPickAndPlace with seed 42, the verifier rejected the first approximately 80 VLM calls at a 100% rate, which from the gradient’s perspective is equivalent to SAC+HER with additional VLM-API cost. The regime ends once the policy begins bringing the gripper close to the object: snapshots fall closer to the goal, the verifier begins admitting relabels, and the small number of admitted trajectories provide low-variance, physics-consistent learning signal.

Throughout this work, we use a window-kernel expectation as the trajectory-conditional re-weighting factor in Semantic PER and directly multiply it into the standard PER priority. Although it works well in our tasks, it may experience difficulty for tasks that involve more distinctive credit-assignment multi-modalities. More expressive priority formulations, such as Gaussian-process posteriors or

learned attention kernels, might be useful for these tasks. Our IS-correction is conservative: we under-correct with the standard PER IS weight rather than the full Semantic-PER IS weight, trading a bounded bias for variance reduction. The fully-corrected ablation, which maintains the Semantic-PER normalizer at sampling time, is left for follow-up work.

Our data verification system is based on simulator forks. Although it provides physics-consistent acceptance gating with zero modeling error, it may require more compute during training data collection since the verifier needs to maintain a parallel MuJoCo state for each VLM call. Extending to real-robot or non-resettable environments would require a learned dynamics-model verifier, with modeling error bounded but no longer zero. A faithful VLM-RB [17] reproduction skeleton accompanies the supplementary material, with a direct head-to-head comparison left for future work. The Oracle-CF kill at the 250k horizon (§4.4) caps the credit-assignment headroom on Fetch at this training budget: if a perfect-information oracle cannot beat HER by $+0.10$, no realistic VLM is expected to do so either. We pre-register an extension to harder goal-conditioned families such as AntMaze and Adroit, where HER’s achieved-future relabel is less effective.

Contributions

- **Daniel Grant.** Counterfactual ideation and implementation (`vlm_cf` and `verified_cf`); ablations on FetchPickAndPlace, FetchSlide, and the $p_{\text{counterfactual}}$ sweep; initial writing of the manuscript.
- **Matei Gardea.** Initial project idea; implementation of an alternative pointwise/pairwise VLM-replay-buffer rescaling combined with PER (a non-positive variant reported as a negative finding); auxiliary 60-run MetaWorld replay-mechanism ablation (cross-benchmark corroboration of $\text{PER} > \text{Uniform}$; reported separately in the extended preprint); editing of the final submission document.
- **Parshawn Gerafian.** Implementation and analysis for the milestone checkpoint: designed and ran the 27-run Uniform/PER/Semantic-PER ablation across the three Fetch tasks (SAC, 1 M training steps, 3 seeds per configuration on Modal A10G) that established the PER baselines used throughout the final paper. Identified the additive-blending failure mode ($0.7 \cdot \text{semantic} + 0.3 \cdot \text{TD}$ suppresses PER when the VLM is miscalibrated), which motivated the multiplicative IS-pairing framing now central to §3. Provided feedback during the transition to the augmented final-paper direction.

References

- [1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] Caleb Chuck, Fan Feng, Carl Qi, Chang Shi, Siddhant Agarwal, Amy Zhang, and Scott Niekum. Null counterfactual factor interactions for goal-conditioned reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025.
- [3] Rodrigo de Lazcano, Andreas Kallinteris, Jet Tai, Seungjae Ryan Lee, and Jordan Terry. Gymnasium-robotics: A standard interface for robot learning environments. <https://robotics.farama.org/>, 2024. Farama Foundation.
- [4] Liang Ding. AgentHER: Hindsight experience replay for LLM agent trajectory relabeling. *arXiv preprint arXiv:2603.21357*, 2026.
- [5] Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018.
- [6] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.

- [7] Michael Y. Hu, Benjamin Van Durme, Jacob Andreas, and Harsh Jhamtani. Sample-efficient online learning in LM agents via hindsight trajectory rewriting. *arXiv preprint arXiv:2510.10304*, 2026.
- [8] Olivia Y. Lee, Annie Xie, Kuan Fang, Karl Pertsch, and Chelsea Finn. Affordance-guided reinforcement learning via visual prompting. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.
- [9] Xing Lei, Wenyan Yang, Kaiqiang Ke, Shentao Yang, Xuetao Zhang, Joni Pajarinen, and Donglin Wang. GCHR: Goal-conditioned hindsight regularization for sample-efficient reinforcement learning. *arXiv preprint arXiv:2508.06108*, 2025.
- [10] Thomas Mesnard, Théophane Weber, Fabio Viola, Shantanu Thakoor, Alaa Saade, Anna Harutyunyan, Will Dabney, Tom Stepleton, Nicolas Heess, Arthur Guez, et al. Counterfactual credit assignment in model-free reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2021.
- [11] Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [12] Shivansh Patel, Xinchun Yin, Wenlong Huang, Shubham Garg, Hooshang Nayyeri, Li Fei-Fei, Svetlana Lazebnik, and Yunzhu Li. A real-to-sim-to-real approach to robotic manipulation with VLM-generated iterative keypoint rewards. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [13] Matthias Plappert, Marcin Andrychowicz, Alex Ray, Bob McGrew, Bowen Baker, Glenn Powell, Jonas Schneider, Josh Tobin, Maciek Chociej, Peter Welinder, Vikash Kumar, and Wojciech Zaremba. Multi-goal reinforcement learning: Challenging robotics environments and request for research. *arXiv preprint arXiv:1802.09464*, 2018.
- [14] Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [15] Erdi Sayar, Zhenshan Bing, Carlo D’Eramo, Ozgur S. Oguz, and Alois Knoll. Contact energy based hindsight experience prioritization. *arXiv preprint arXiv:2312.02677*, 2023.
- [16] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- [17] Elad Sharony, Tom Jurgenson, Orr Krupnik, Dotan Di Castro, and Shie Mannor. VLM-guided experience replay. *arXiv preprint arXiv:2602.01915*, 2026.
- [18] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [19] Núria Armengol Urpí, Marco Bagatella, Marin Vlastelica, and Georg Martius. Causal action influence aware counterfactual data augmentation. *arXiv preprint arXiv:2405.18917*, 2024.
- [20] Rui Zhao and Volker Tresp. Energy-based hindsight experience prioritization. In *Conference on Robot Learning (CoRL)*, 2018.

A Method Details and Environments

Cross-task VLM prompt template. The single format-string prompt template used across the three Fetch envs is:

```
You are an expert robotics analyst. The robot is attempting the
following task: {task_description}. Examine the {K} keyframes of
a failed episode shown below. Identify the single keyframe index
in {0,...,K-1} at which the failure became inevitable, and a 1-2
sentence reasoning. Return strict JSON: {"failure_frame_index": i,
"reasoning": s}.
```

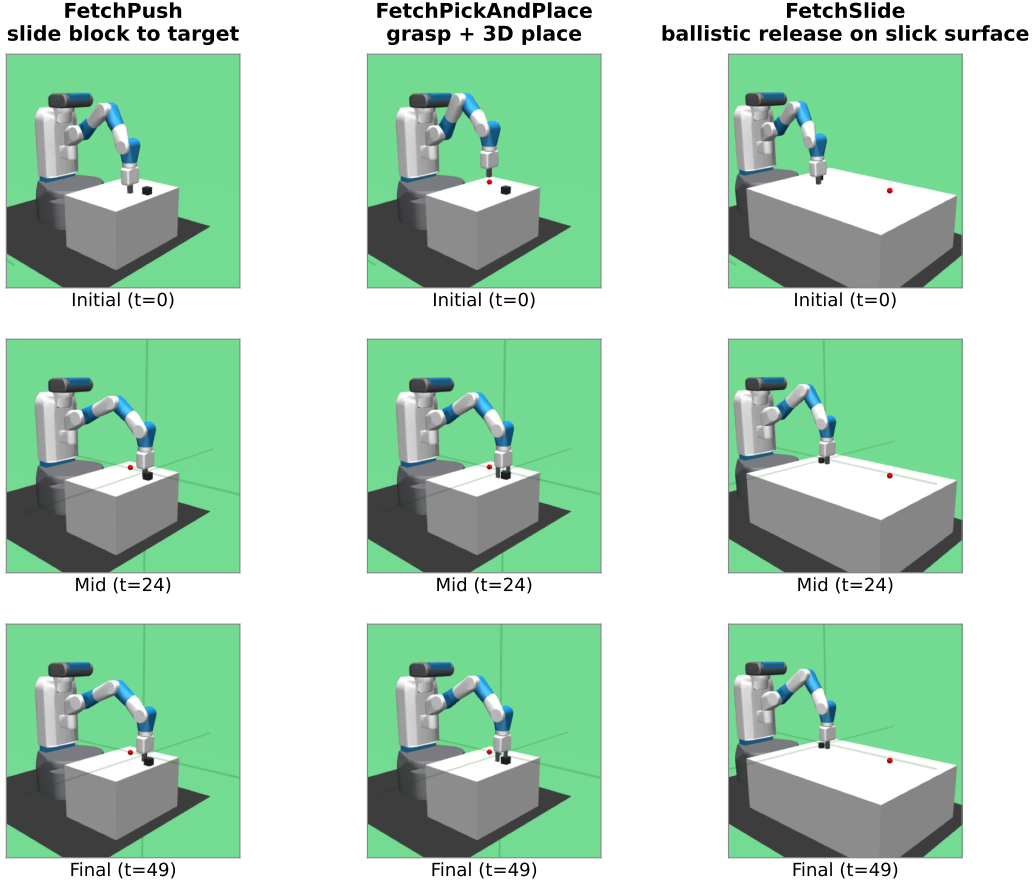


Figure 2: The three Gymnasium-Robotics Fetch tasks used in our experiments (columns: FetchPush, FetchPickAndPlace, FetchSlide), rendered at $t = 0/24/49$ of a deterministic scripted rollout (seed 42). Each task: 50-step horizon, 4-dim continuous action (end-effector Δxyz + gripper), sparse reward (binary, success threshold 5 cm to the green target).

Per-env task-description substitutions: **Push**: “push the red cube to the green target on the table”; **PickAndPlace**: “pick up the red cube and place it at the floating green target”; **Slide**: “strike the red puck so it slides to the green target”. The verified-CF mechanism uses an ACTION variant of this template eliciting a 4-vector action sequence; full prompt texts and the localizer code accompany the report.

Heuristic localizer (Oracle v3). GoalDistanceLocalizer reads privileged sim state and picks the failure timestep in three phases, applied in precedence order. **(a) ballistic**: the latest contiguous run of ≥ 3 steps with post-peak object velocity > 0.05 m/step (Slide free-flight). **(b) contact-loss**: the latest transition $\text{in_contact}_t \rightarrow \neg \text{in_contact}_{t+1}$ with run length ≥ 2 and object-goal distance > 0.10 m (Push and PnP drop-offs). **(c) argmin**: closest-approach timestep, as a fallback. The localizer plays two roles — it supplies a supervised transfer target, and it gives Semantic PER its privileged-state upper-envelope curve.

B Bias-Bound Proof Sketch

Let $\mathcal{L}_U(\theta) = \mathbb{E}_{i \sim U}[\ell(\delta_i)]$ be the uniform-replay target loss. Standard PER with annealed correction yields, in the $\beta_t \rightarrow 1$ limit, an unbiased self-normalized IS estimator of $\mathcal{L}_U$ [16, Sec. 3.4]. Our Semantic PER proposal $\mu_{\text{Sem}} \propto \mu_P \cdot w_{\text{sem}}$ is a re-weighting of μ_P by a trajectory-conditional factor bounded by $1 \leq w_{\text{sem}} \leq w_{\text{max}}$. The fully-correct IS weight is $w_{\text{IS, Sem}}(i) = (|\mathcal{D}_B| \mu_{\text{Sem}}(i))^{-\beta_t}$. Our

Table 2: Main hyperparameters. Full per-table breakdowns (SAC architecture, HER strategy, PER schedule, Semantic-PER, Verified-CF, VLM call) are listed in the supplementary configuration files.

Parameter	Value	Notes
<i>SAC</i> [6]		
Hidden layers $\times$ width	2×256 , ReLU	Twin-Q; tanh-Gaussian actor
LR (actor/critic/ α)	3×10^{-4}	Adam, default $(\beta_1, \beta_2, \varepsilon)$
Discount γ / Polyak τ	0.98/0.005	Fetch-standard
Target entropy $\bar{H}$	$-d_a = -4$	Auto-tuned
Batch / updates per step	256/1	10k warm-up uniform-random
<i>HER</i> [1]		
Strategy / k_{HER}	<code>future</code> / 4	4 hindsight transitions per real
<i>PER</i> [16]		
α / $\beta_0 \rightarrow \beta_{\text{end}}$	0.6/0.4 $\rightarrow$ 1.0	Linear over 500k steps; $\varepsilon = 10^{-6}$
<i>Semantic PER</i>		
Window half-width W	5 timesteps	11-slot box around t_{fail}
Boost cap w_{max}	10	Multiplicative factor on PER priority
Keyframes K / VLM provider	5 / GPT-4o (deployed)	<code>gpt-4o-2024-08-06</code>
<i>Verified-CF</i>		
Verifier rollout horizon N	50 steps	One Fetch episode
Success threshold η / teleport reject ρ	0.05/0.05 m	Env’s native threshold
CF prompt variant	ACTION	Structurally teleport-immune
<i>Training</i>		
Total env steps	500k (primary)	250k–1M for selected ablations
Random seeds	42, 123, 999	3 per (method, env) cell
Replay capacity	10^6 slots	Ring buffer with sum-tree priorities

implementation under-corrects with $w_{\text{IS},P}$ (a V-trace-style [5] variance-reduction choice, akin to truncated importance ratios in Retrace, 11). The per-slot ratio is $w_{\text{IS},P}/w_{\text{IS},\text{Sem}} = w_{\text{sem}}^{\beta_t}/Z^{\beta_t}$ where Z is the μ_{Sem} normalizer. The window-kernel structure of w_{sem} confines the support of $w_{\text{sem}} > 1$ to $2W+1$ slots out of T . Combining these gives

$$|\hat{\mathcal{L}}_{\text{under}} - \mathcal{L}_U| \leq C (w_{\text{max}}^{\beta_t} - 1) \frac{2W+1}{T} \|\ell\|_{\infty}, \quad (2)$$

with C a constant depending on $|\mathcal{D}_B|$ and the PER normalization. At our defaults ($w_{\text{max}} = 10$, $W = 5$, $T = 50$, $\beta_t \leq 1$) the bias multiplier is at most $(10-1) \cdot 11/50 \approx 1.98 C \|\ell\|_{\infty}$, and $\rightarrow 0$ as $w_{\text{max}} \rightarrow 1$. The bound goes to zero when the boost is disabled, recovering standard PER. The fully-corrected ablation (run $w_{\text{IS},\text{Sem}}$) removes the residual bias entirely at the cost of maintaining Z . **Two-regime degeneracy.** On the verifier channel, miscalibration acts on $m_{\phi}(\tau) = m_{\text{gen}} \cdot m_{\text{ver}}(\sigma)$ (VLM parse rate $\times$ verifier acceptance rate); the $\varepsilon_{\text{cal}} = 0$ guarantee holds vacuously when the accepted set is empty (cold-start: $m_{\text{ver}} \rightarrow 0$). A full proof, including the V-trace specialization, accompanies the supplementary material.

C Hyperparameters and Compute

Compute. Primary runs use a single NVIDIA RTX 5070 Ti (16 GB); Modal-cloud seeds use one L4 (24 GB) per run. Wall-clock per 500k-step run is roughly 6h for SAC+HER and 9h for SAC+HER+VLM-CF, with the extra three hours dominated by VLM-API latency rather than compute. The project total comes to ~ 220 GPU-hours plus $\sim \$80$ in VLM-API spend. Verifier snapshot round-trip latency is 17.6 ms per call (0.4% overhead at ~ 1700 failed episodes per 100k steps). Reproduction commands, W&B run-IDs, and per-seed eval traces accompany the report.

D Additional Experimental Detail

Real-data validation (extended). The $n = 10$ real-failure set is drawn from failed eval episodes of partially-trained SAC+HER checkpoints (PickAndPlace at 50k steps and Push at 30k steps, both near

5% success), with final-distance in $[0.06, 0.34]$ m on Push and a median of 0.27 m on PickAndPlace. Three findings carry over from the synthetic set: **(i)** teleport-collapse is task-conditioned (Push: 100%/75% for Opus/Sonnet on ACHIEVED_GOAL; PickAndPlace: 75%/0%, with Sonnet instead proposing a waypoint ~ 8 cm above the goal); **(ii)** the 5 cm reject-teleport gate fires on 12/20 position-emitting generations across both vendors (cross-vendor floor on Push: 75%), and the action-emitting verifier sidesteps the issue entirely; **(iii)** plausibility shifts by only ± 0.07 from synthetic to real, while the ACTION sign-flip rate worsens (Opus $0.50 \rightarrow 0.55$; Sonnet 0.70).

Statistical methodology and cross-task notes. Each (method, env) cell uses $n = 3$ seeds (42, 123, 999), reported as mean $\pm$ SE with the t -distribution at $n - 1 = 2$ d.f.; comparisons rely on non-overlapping SE intervals, a conservative choice at this sample size. The pivotal 0/6 vs. 4/4 contrast in Table 1 is decided by a Euclidean predicate, and Fisher’s exact test rejects equal proportions at $p < 0.005$. The 12-episode pilot uses 4 episodes per env, with the VLM correctly identifying task-specific failure modes in every case; the 2/4 PickAndPlace keyframe-agreement reflects heuristic under-localization rather than VLM error (closed-loop grip timing is the hardest mode for geometry-only oracles [15]).

E Broader Impacts and Reproducibility

This is methodological, simulation-only work; we make no real-robot or sim-to-real claims. On the positive side, fewer environment steps reduce GPU-hours, the IS-posterior framing offers a principled lens on VLM-in-replay, and the verified-CF gate is a transferable “language-prior + physics-test” pattern. On the negative side, the VLM-API energy and dollar costs are non-trivial (per-run spend is reported, and a heuristic Oracle fallback is available). Code, configs, prompt texts, the V-trace specialization, the two-regime q_ϕ proof, W&B run-IDs, and a VLM-RB [17] reproduction skeleton accompany the report.